GTSRB Traffic Sign Recognition Using Deep Learning

PROJECT SYNOPSIS

1. Project Title

GTSRB Traffic Sign Recognition Using Deep Learning

2. Problem Statement

Traffic signs communicate rules and warnings, but recognition is difficult when signs are small, tilted, blurred, unevenly illuminated, or visually similar to other classes. The German Traffic Sign Recognition Benchmark contains 43 categories and real-world variation, making it a useful computer-vision classification problem. The project addresses the classification task: given one cropped traffic-sign image, predict its class and confidence. Full-scene object detection and safety-critical driving decisions are outside the scope of this version.

3. Project Abstract

This project builds a reproducible TensorFlow/Keras pipeline for the 43-class German Traffic Sign Recognition Benchmark. The complete archive is downloaded from Zenodo, verified with the published MD5 checksum, and checked for every class ID from 0 through 42. The pipeline performs exploratory data analysis, grouped validation splitting, resizing, normalization, moderate training-only augmentation, model training, official-test evaluation, error analysis, Grad-CAM visualization, and single-image inference. A compact custom CNN trained for 30 epochs on a Colab GPU achieved 97.12% test accuracy and 95.14% macro F1. A frozen-backbone MobileNetV2 comparison trained for 25 epochs achieved 76.18% accuracy and 68.23% macro F1. The result is an auditable classification system with documented limitations and reproducible artifacts.

4. Literature Survey

The project design is based on verified primary papers and official documentation. Stallkamp et al. define GTSRB as a multi-category traffic-sign benchmark with changes in distance, illumination, weather, occlusion, and rotation; this motivates the dataset, EDA, and augmentation strategy. Sandler et al. describe MobileNetV2's inverted residuals, linear bottlenecks, and depthwise convolutions; this motivates a lightweight transfer-learning comparison. Kingma and Ba introduce Adam, used as the practical baseline optimizer. Selvaraju et al. introduce Grad-CAM, used to visualize regions contributing to a prediction. Official scikit-learn, TensorFlow, and Keras documentation support grouped validation, streaming input pipelines, augmentation, early stopping, learning-rate reduction, and checkpoint saving.

Source	Verified contribution	Project use
Stallkamp et al. 2011	GTSRB benchmark and visual variation	Dataset boundary and evaluation
Sandler et al. 2018	Efficient MobileNetV2 architecture	Transfer-learning comparison
Kingma and Ba 2015	Adaptive Adam optimizer	Training optimizer
Selvaraju et al. 2017	Gradient-based visual explanations	Grad-CAM diagnostic
Official documentation	Grouped split, tf.data, augmentation, callbacks	Validation and pipeline design

5. System Specifications

The final benchmark was executed in Google Colab with a visible GPU. Local development and inference were also verified on an Apple M1 environment. Dataset and trained models remain outside Git because they are generated artifacts.

Parameter	Specification	Operational purpose
Benchmark hardware	Google Colab GPU	30/25-epoch training and evaluation
Local hardware	Apple M1, 8 GB unified memory	Development, smoke tests, inference
Dataset	39,209 train; 12,630 official test	43-class recognition benchmark
Storage policy	Dataset and artifacts ignored by Git	Keep repository reproducible and lightweight

6. Software and Environment Specifications

Software / library	Version / platform	Role
Python	3.11 project environment	Reproducible execution
TensorFlow / Keras	GPU-capable Colab runtime	Training and inference
NumPy / pandas	Pinned project dependencies	Arrays and label tables
scikit-learn	GroupShuffleSplit and metrics	Validation and evaluation
Matplotlib / Pillow / OpenCV	Pinned project dependencies	EDA, images, plots, Grad-CAM

7. Methodology

The archive is downloaded and verified before extraction. Training labels are inferred from class directories, while official test labels are read from Test.csv. Related sequence-like identifiers are grouped before creating a validation split to reduce near-duplicate leakage. Images are resized to 64 × 64 for the custom CNN and 128 × 128 for MobileNetV2, converted to RGB, and normalized to [0, 1]. Rotation, translation, zoom, contrast, and brightness transformations are applied only during training.

Stage	Implementation	Reason
Acquisition	Zenodo download plus MD5 verification	Trust the input archive
EDA	Class counts, sample grid, dimensions, brightness, color	Understand data variation
Validation	Grouped 85/15 split from training data	Reduce sequence leakage
Baseline	Compact CNN from scratch	Transparent reference model
Comparison	ImageNet MobileNetV2 with frozen backbone	Lightweight transfer-learning baseline
Training control	Adam, early stopping, LR reduction, checkpoint	Stable and recoverable training
Interpretability	Grad-CAM over a final convolutional layer	Inspect image regions used

8. Expected and Measured Outputs

Every run saves a Keras model, training history, learning curves, classification report, confusion matrix, and error-analysis plot. The dashboard summarizes EDA and model comparison. The standalone Colab notebook downloads the dataset directly, trains both models with live GPU output, and packages the resulting models and reports into a downloadable ZIP.

Model	Input / run	Measured official-test result
Custom CNN	64 × 64; 30 Colab GPU epochs	Accuracy 97.12%; Macro F1 95.14%
MobileNetV2	128 × 128; 25 Colab GPU epochs	Accuracy 76.18%; Macro F1 68.23%

9. Scope and Limitations

The result is a classification system for cropped sign images, not a complete autonomous-driving component. The custom CNN currently outperforms the frozen MobileNetV2 baseline in the Colab run; MobileNetV2 should be fine-tuned and retuned before drawing broader architectural conclusions. The optional YOLO code is a future detector scaffold because the classification archive does not include full-scene bounding-box annotations. Further work should include calibration, out-of-distribution detection, longer matched experiments, fine-tuning, and real-world safety validation.

10. Project Deliverables

The repository contains the verified downloader, reusable TensorFlow modules, command-line training/evaluation/inference interfaces, EDA and reporting scripts, tests and CI configuration, a standalone Colab notebook, an interactive dashboard, optional Grad-CAM and YOLO extensions, an IEEE-style report, a presenter guide, and an IEEE presentation with speaker notes.

Deliverable	Location	Purpose
Colab notebook	notebooks/traffic_sign_recognition_colab.ipynb	Standalone GPU execution
Project report	docs/traffic_sign_recognition_project_report.pdf	Detailed technical record
Synopsis	docs/traffic_sign_recognition_synopsis.pdf	Short project summary
Presenter guide	docs/traffic_sign_recognition_presenter_guide.pdf	Beginner explanation and viva preparation
Presentation	docs/traffic_sign_recognition_ieee_presentation.pptx	Presentation with speaker notes
Literature survey	docs/literature_survey.md	Verified sources and design mapping

11. Conclusion

This project demonstrates the complete machine-learning lifecycle for traffic-sign recognition: verified data acquisition, exploratory analysis, leakage-aware validation, preprocessing, model training, GPU benchmarking, multi-metric evaluation, visual explanation, and communication. The Colab result shows that a well-trained compact CNN can reach high accuracy on this classification benchmark, while the weaker MobileNetV2 result identifies a concrete next experiment: fine-tune the transfer-learning backbone instead of leaving it frozen.

12. References

[1] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, The German Traffic Sign Recognition Benchmark: A multi-class classification competition, IJCNN, 2011. https://www.ini.rub.de/upload/file/1470692848_f03494010c16c36bab9e/StallkampEtAl_GTSRB_IJCNN2011.pdf

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, MobileNetV2: Inverted Residuals and Linear Bottlenecks, CVPR, 2018. https://arxiv.org/abs/1801.04381

[3] D. P. Kingma and J. Ba, Adam: A Method for Stochastic Optimization, ICLR, 2015. https://arxiv.org/abs/1412.6980

[4] R. R. Selvaraju et al., Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization, ICCV, 2017. https://openaccess.thecvf.com/content_ICCV_2017/papers/Selvaraju_Grad-CAM_Visual_Explanations_ICCV_2017_paper.pdf

[5] scikit-learn, GroupShuffleSplit documentation. https://scikit-learn.org/1.5/modules/generated/sklearn.model_selection.GroupShuffleSplit.html

[6] TensorFlow and Keras official documentation for tf.data, preprocessing layers, and callbacks. https://www.tensorflow.org/guide/data_performance and https://keras.io/api/callbacks/